LESSON 16 · CHEAT SHEET

Records, Sealed Types & Patterns

Java 16 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. Records: validate and normalise in the **compact constructor**. 16
2. One record per alternative; close the set with `sealed … permits`. 17
3. Switch over a sealed type: **every case, no default**.
4. Specific (guarded) cases **before** general ones (dominance).
5. `case null` when null is possible. 21
6. Record `equals` uses component equals: BigDecimal scale matters.
7. `getFirst/getLast/reversed` (a view) over index maths. 21

Canonical snippets

1 · An algebraic data type

```
sealed interface PaymentResult
    permits Approved, Declined, Pending, Refunded {}
record Approved(String id, String m, BigDecimal amt)
    implements PaymentResult {}
```

2 · Pattern switch with a guard

```
switch (r) {
  case Approved(var i, var m, var a) when big(a) -> ..
  case Approved x -> ..
  case Declined d -> ..   // … no default
}
```

3 · instanceof pattern in a stream

```
.map(r -> r instanceof Approved a ? a.amount() : ZERO)
```

Version map

Feature	Final
switch expressions (arrow, yield)	14
text blocks	15
records, local records	16
pattern matching for instanceof	16
sealed classes / interfaces	17
pattern matching for switch, case null	21
record patterns (nested), guards	21
SequencedCollection / SequencedMap	21
unnamed patterns _	22: Beyond Java 21

Sequenced collections (real output)

```
payments.getFirst() / getLast()  P001 / P012
payments.reversed() first 3      [P012, P011, P010]
monthly.firstEntry()             2026-08=3314.36
LinkedHashMap.pollFirstEntry()   P003=PAYPAL
```

Mini diagram: ADT



Real results from the course code

Call	Result
`Money.of("SGD", "10.005")`	SGD 10.01
SGD plus USD	currency mismatch: SGD vs USD
count by kind	{APPROVED=8, DECLINED=2, PENDING=1, REFUNDED=1}
`describeJava21(null)`	null!

Worked example / exercise	Result
outcome report, 3 versions	… settled=2999.19 (identical)
claims by kind	{APPROVED=3, PARTIAL=2, REJECTED=2, UNDER_REVIEW=1}
claims total paid	35280.50

Anti-patterns → fixes

Don't	Do
status flag + nullable fields	one record per alternative
`default` over a sealed type	list every case
final `else` catch-all	exhaustive switch expression

Don't	Do
general case before guarded	specific cases first
raw BigDecimal in a record	normalise scale in constructor
`list.get(list.size() - 1)`	`list.getLast()` 21

Lesson 16 · sticky notes

Cut out, stick on your wall, and read one every morning.

Compact constructor

validate · normalise
List.copyOf

Sealed + records

= algebraic data type
exactly one of …

No default

over a sealed type:
let the compiler check

Record pattern

case Approved(var id,
var m, var amt) ->

Guards

case X x when cond
specific cases first!

instanceof [16]

o instanceof Customer c
&& c.tier() == …

case null [21]

handle null in the
switch, not before it

Sequenced [21]

getFirst · getLast
reversed() is a view